

Purpose of this Handbook

This handbook is a **practical companion** to the Software Engineering course. It is designed to support **active, experiential, and reflective learning** during class time.

Rather than using class sessions primarily for lecture, this course emphasizes:

- Hands-on activities
- Team-based problem solving
- Communication and collaboration
- Reflection on process, not just product

Each week includes:

- A **brief overview** outlining learning goals and pacing notes
- A **menu of in-class activities** aligned to those goals
- Individual activity pages with space for **instructor notes and iteration**

This structure allows instructors to:

- Select activities that fit their teaching style
 - Adapt pacing based on class size and constraints
 - Evolve the course over time while preserving intent
-

How to Use This Handbook

- Each **week overview** provides context, objectives, and outside-of-class work
- Each **activity has its own page**, intended to be printed and annotated
- Activities are a **menu**, not a checklist — instructors should choose what best fits their class
- Notes sections are intentionally left open for reflection and improvement

This handbook is meant to be **used, marked up, and revised**.

Pedagogical Commitments

This course prioritizes:

- Psychological safety and respectful collaboration
- Clear communication and shared responsibility

- Ethical and reflective software practice
- Learning through iteration, feedback, and failure

Students are expected to engage actively, contribute thoughtfully, and support one another's learning.

Notes for Instructors

This handbook is intentionally **not prescriptive**.

If something doesn't work for your class:

- Change it
- Replace it
- Make it better

Please record what you try and what you learn — future instructors (including you) will thank you.

Last updated: December 2025

Course offering: Winter 2026

Author(s): Himbeault L.

Contents

Purpose of this Handbook	1
How to Use This Handbook	1
Pedagogical Commitments	1
Notes for Instructors	2
Week 1 - Software Development Models & Agile Mindset	6
Week overview	6
Learning objectives	6
Outside-of-class work	6
Pacing and sequencing notes	6
Instructor notes	7
Activity 1 Marshmallow Challenge (Agile Edition)	8
Activity 2 Agile vs. Waterfall Simulation	9
Activity 3 SDLC Stage Sort	10
Activity 4 Agile Values Mismatch	11
Activity 5 Vision Elevator Pitch	12

Week 2 - Planning Artifacts & Project Architecture	13
Week overview	13
Learning objectives	13
Outside-of-class work	13
Pacing and sequencing notes (important)	13
Instructor notes	14
Activity 1 - Team Formation & Working Agreement (Team Canvas)	15
Activity 2 - Architecture Diagram Charrette	16
Activity 3 - Impact-Effort Matrix for Features	17
Activity 4 - GenAI Prompt Brainstorm (Responsible AI Use)	18
Week 3 - Version Control & Testing Foundations	19
Week overview	19
Learning objectives	19
Outside-of-class work	19
Pacing and sequencing notes (important)	20
Instructor notes	20
Activity 1 - Git Team Scavenger Hunt	21
Activity 2 - Branching Model Simulation (Human Git Tree)	22
Activity 3 - Merge Conflict Race	23
Activity 4 - Testing Pyramid Quiz & Discussion	24
Activity 5 - Test Case Design Relay	25
Activity 6 - TDD Ping-Pong Pairing	26
Week 4 - Exceptional Testing & Technical Debt	27
Week overview	27
Learning objectives	27
Outside-of-class work	27
Deliverables	28
Pacing and sequencing notes (important)	28
Instructor notes	28
Activity 1 - Edge Case Brainstorm (Bug Hunt)	29
Activity 2 - Mock Objects Role-Play	30
Activity 3 - Technical Debt Jenga	31
Activity 4 - Technical Debt “Investment” Discussion	32
Activity 5 - Consensus Conflict Resolution Exercise	33
Activity 6 - Five Whys Drill-Down	34
Week 5 - Design Principles & Refactoring	35
Week overview	35
Learning objectives	35
Outside-of-class work	35
Pacing and sequencing notes (important)	36
Instructor notes	36
Activity 1 - Code Smells Scavenger Hunt Session	37

Activity 2 - SOLID Design Poker Session	38
Activity 3 - Refactoring Golf	39
Activity 4 - Mob Programming Refactor Session	40
Activity 5 - AI Pair-Programming Challenge	41
Week 6 - Alternatives to Inheritance & Dependency Injection	42
Week overview	42
Learning objectives	42
Outside-of-class work	42
Pacing and sequencing notes (important)	43
Instructor notes	43
Activity 1 - Inheritance vs. Composition Debate	44
Activity 2 - Strategy Pattern Role-Play	45
Activity 3 - Refactor Inheritance to Composition Exercise	46
Activity 4 - Dependency Injection Kata	47
Activity 5 - Error-Handling Hot Potato	48
Activity 6 - Exception Testing Challenge	49
Activity 7 - Midterm Retrospective (Team Check-In)	50
Suggested at-home practice activities	51
Week 7 - Design Patterns & Architecture	52
Week overview	52
Learning objectives	52
Outside-of-class work	52
Pacing and sequencing notes (important)	53
Instructor notes	53
Activity 1 - Pattern Charades	55
Activity 2 - Real-World Pattern Hunt	56
Activity 3 - Architecture Kata (Mini)	57
Activity 4 - MVC Role-Play	58
Activity 5 - Design Pattern Show and Tell	59
Activity 6 - Pattern-Based Code Refactoring	60
Activity 7 - Architecture Comparison Discussion	61
Week 8 - Advanced Testing	62
Week overview	62
Learning objectives	62
Outside-of-class work	62
Pacing and sequencing notes (important)	63
Instructor notes	63
Activity 1 - Continuous Integration Pipeline Race	64
Activity 2 - Microservice Integration Simulation	65
Activity 3 - Test Doubles Showdown	66
Activity 4 - Flaky Test Roulette	67
Activity 5 - Acceptance Criteria Workshop	68

Activity 6 - Bug Triage Meeting (Role-Play)	69
Activity 7 - Continuous Testing with AI Pairing	70
Week 9 - Legacy Software, Architecture Recovery & Change	71
Week overview	71
Learning objectives	71
Outside-of-class work	71
Pacing and sequencing notes (important)	72
Instructor notes	72
Activity 1 - “Swap an Old Assignment” Legacy Exercise	73
Activity 2 - Legacy Code Scavenger Hunt	74
Activity 3 - Strangler Fig Refactoring Plan	75
Activity 4 - “Telephone” Code Communication	76
Activity 5 - Legacy Refactoring Golf (Coding Dojo)	77
Activity 6 - Write the Missing Documentation	78
Activity 7 - Regression Testing Strategy Challenge	79
Activity 8 - Legacy System Case Study Discussion	80
Activity 9 - Museum of Antiquated Code	81
Week 10 - Measuring Team and Project Successes	82
Week overview	82
Learning objectives	82
Outside-of-class work	82
Pacing and sequencing notes (important)	82
Instructor notes	83
Activity 1 - Project Showcase and Demo Reel Debrief	84
Activity 2 - Burndown Chart Analysis Game	85
Activity 3 - Team Retro - Start, Stop, Continue	86
Activity 4 - Agile Values Retrospective Sort	87
Activity 5 - Roles in Software Engineering Pictionary	88
Activity 6 - Kudos and Specific Appreciation Session	89

Week 1 - Software Development Models & Agile Mindset

Week overview

This week introduces students to software development as a **process**, rather than a purely technical activity. The focus is on surfacing students' existing mental models, introducing common process frameworks, and framing Agile as a set of values and principles rather than a prescriptive methodology.

Students are **not expected to master** any development methodology this week. The goal is shared vocabulary, comparison, and reflection that will support later project work.

Learning objectives

By the end of this week, students should be able to:

Process and Practice

- Describe software development as an iterative process
 - Compare common process models (e.g., Waterfall and Agile)
 - Articulate the values and principles behind Agile development
 - Reflect on how process choices affect team behavior and outcomes
-

Outside-of-class work

Readings

- A Beginner's Guide to Software Development Life Cycle (SDLC)
- Agile vs. Waterfall: Which Is Right for Your Project?
- Manifesto for Agile Software Development
- Becoming a 10x Developer (Kate Heddlestone)

Deliverable

- Get-to-know-you survey (used later for group formation)
-

Pacing and sequencing notes

Week 1 has **reduced in-class activity time** due to syllabus review and course orientation.

- The first class prioritizes alignment, expectations, and shared framing
- Remaining class time shifts into experiential, activity-based learning

Instructors should expect to run **23 activities total**. Activities are a **menu**, not a checklist.

Instructor notes

Students often:

- Arrive with strong opinions about “best” methodologies
- Conflate Agile with specific practices (stand-ups, sprints)
- Have uneven prior exposure to software process

Encourage disagreement and uncertainty. Ambiguity is productive at this stage.

Activity 1 Marshmallow Challenge (Agile Edition)

Learning focus: iteration, experimentation, planning assumptions

Purpose

Experience the difference between heavy upfront planning and iterative experimentation.

Learning objectives targeted

- Compare and contrast software development methodologies
- Describe agile values and principles

Team focus

Collaboration, experimentation, communication

Recommended group size

4 students

Time estimate

~35–40 minutes

Best for

Activity-flexible time

Description

Teams build the tallest free-standing structure using spaghetti, tape, string, and a marshmallow. Measurement and debrief connect outcomes to Agile vs. Waterfall assumptions.

Debrief prompts

- When did you first test the marshmallow?
- How did planning vs. experimentation affect results?

Notes / Reflections

[illegible]

Activity 2 Agile vs. Waterfall Simulation

Learning focus: trade-offs, adaptability, constraints

Purpose

Make trade-offs between methodologies visible through experience.

Learning objectives targeted

- Compare and contrast software development methodologies
- Understand the foundations of software engineering process

Team focus

Decision-making, adaptability

Recommended group size

Large groups or multiple medium groups

Time estimate

25–30 minutes

Best for

Activity-flexible time

Description

Two groups complete the same task under different process constraints (Agile vs. Waterfall), followed by a comparison of outcomes.

Debrief prompts

- Which approach adapted more effectively?
- When might each approach be appropriate?

Notes / Reflections

Activity 3 SDLC Stage Sort

Learning focus: SDLC stages, sequencing, overlap

Purpose

Clarify SDLC stages and challenge linear assumptions.

Learning objectives targeted

- Understand the foundations of software engineering process
- Compare traditional and agile approaches

Team focus

Shared understanding, negotiation

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Groups sort software activities into SDLC phases and discuss how Agile reorders or overlaps these stages.

Debrief prompts

- Which tasks were hardest to place?
- How does Agile change this structure?

Notes / Reflections

[illegible]

Activity 4 Agile Values Mismatch

Learning focus: values, judgment, ambiguity

Purpose

Practice identifying Agile values in concrete situations.

Learning objectives targeted

- Describe agile values and principles

Team focus

Interpretation, discussion

Recommended group size

Any

Time estimate

15–20 minutes

Best for

Lecture-constrained time

Description

Students vote on whether scenarios align more closely with Agile or traditional values, prompting discussion.

Debrief prompts

- Why were some scenarios ambiguous?
- Can Agile values conflict?

Notes / Reflections

[illegible]

Activity 5 Vision Elevator Pitch

Learning focus: clarity, alignment, communication

Purpose

Practice articulating a clear and compelling product vision.

Learning objectives targeted

- Describe agile values and a product vision

Team focus

Communication, confidence

Recommended group size

3–5 students

Time estimate

25–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams craft and present a 30-second product vision pitch and receive feedback.

Debrief prompts

- What made a vision compelling?
- What details were unnecessary?

Notes / Reflections

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

Week 2 - Planning Artifacts & Project Architecture

Week overview

This week moves students from high-level product vision into **concrete planning artifacts** that support implementation. Students learn how non-technical artifacts (vision, features, user stories) connect to technical artifacts (architecture diagrams, release plans), and how these artifacts function as *communication and alignment tools* within Agile teams.

This is also the week where **permanent project teams are formed**. Class time intentionally supports team formation, shared expectations, ethical responsibility, and collaborative planning. Planning is framed not as bureaucracy, but as a way teams make decisions together under uncertainty.

Learning objectives

By the end of this week, students should be able to:

- Describe, create, and apply **non-technical planning artifacts** (e.g., project vision → features → user stories)
- Describe, create, and apply **technical planning artifacts** (e.g., architecture diagrams, release plans)
- Build an architecture diagram for implementing a vision
- Describe the **purpose** of technical planning artifacts
- Prepare a release plan for a project

Outside-of-class work

Readings

- User Stories and User Story Examples by Mike Cohn
- Ultimate Guide to Writing a High-Quality User Story
- Creating a Software Architecture Diagram
- 7 Steps to Create a Successful Release Plan
- The Journey of a Lone Female Software Developer

Pacing and sequencing notes (important)

This week may include both **lecture-constrained time** (introducing planning artifacts, framing group work) and **activity-flexible time**.

- Temporary groupings may be used early in the week
- **Permanent project teams must be formed by the end of the week**

Instructors should plan to include:

- At least **one planning-focused activity**
- At least **one team or group-dynamics activity**

Instructor notes

Students often:

- Jump from vision directly to implementation
- Treat planning artifacts as static documentation
- Underestimate how much planning is about communication, not correctness

Team-focused activities this week are intentionally foundational. Explicit discussion of expectations, responsibility, and ethics early helps prevent conflict later.

Activity 1 - Team Formation & Working Agreement (Team Canvas)

Learning focus: team alignment, responsibility, communication

Purpose

Establish shared expectations, norms, and responsibilities within permanent project teams.

Learning objectives targeted

- Build shared responsibility and accountability
- Practice explicit communication and expectation-setting

Team focus

Trust, communication, ethical responsibility

Recommended group size

Permanent project teams (3–5 students)

Time estimate

45–60 minutes

Best for

Activity-flexible time

Description

Newly formed project teams complete a Team Working Agreement or Team Canvas. Using a provided template, teams define:

- Roles and responsibilities
- Communication channels and response expectations
- Meeting norms
- How conflict and unmet expectations will be addressed
- How peer feedback will be given

Teams should explicitly discuss both shared goals and individual goals.

Debrief prompts

- Which expectations felt obvious? Which did not?
- What kinds of problems is this agreement meant to prevent?
- How should this document evolve over time?

Notes / Reflections

Activity 2 - Architecture Diagram Charrette

Learning focus: architecture as a communication tool

Purpose

Explore architecture diagrams as collaborative thinking artifacts.

Learning objectives targeted

- Build an architecture diagram for implementing a vision
- Describe the purpose of technical planning artifacts

Team focus

Perspective-taking, communication

Recommended group size

3–5 students

Time estimate

35–45 minutes

Best for

Activity-flexible time

Description

Teams sketch a high-level architecture diagram for a project or case study. Diagrams are then rotated between teams in a gallery walk. Each visiting team adds one question, concern, or improvement.

Original teams revise their diagram before briefly presenting it.

Debrief prompts

- What did other teams notice that you missed?
- How did feedback change your design?
- Why is architecture primarily a communication artifact?

Notes / Reflections

Activity 3 - Impact-Effort Matrix for Features

Learning focus: prioritization and value assessment

Purpose

Practice evaluating features based on value versus cost.

Learning objectives targeted

- Describe and apply non-technical planning artifacts
- Prepare for release planning

Team focus

Reasoning, justification

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams place proposed features onto an Impact–Effort matrix and discuss prioritization. Each team identifies one feature to do early and one to defer or drop.

Debrief prompts

- What made a feature “high impact”?
- How did effort estimates influence decisions?
- How does this connect to release planning?

Notes / Reflections

Activity 4 - GenAI Prompt Brainstorm (Responsible AI Use)

Learning focus: ethics, responsibility, tool use

Purpose

Develop shared norms for responsible use of AI tools in project work.

Learning objectives targeted

- Ethical responsibility in software development
- Team communication and accountability

Team focus

Ethics, trust, shared standards

Recommended group size

Permanent project teams

Time estimate

15–25 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams discuss allowed and disallowed uses of GenAI tools for their project, identify risks, and agree on mitigation strategies. Each team records 3–5 concrete guidelines.

Debrief prompts

- What risks concern you most?
- How will your team ensure accountability?
- How does this align with professional responsibility?

Notes / Reflections

Week 3 - Version Control & Testing Foundations

Week overview

This week introduces students to **version control and testing as foundational team practices**, not just technical tools. Students learn how version control supports collaboration, how different workflows shape team behavior, and how testing supports correctness, maintainability, and confidence in change.

Class time emphasizes **hands-on practice and shared problem-solving**. Students will encounter common sources of friction-merge conflicts, coordination breakdowns, unclear expectations-in a low-stakes environment. Testing activities focus on helping students reason about *what* to test and *why*, before focusing on tools or frameworks.

Any additional group members should be introduced by the end of this week.

Learning objectives

By the end of this week, students should be able to:

Tools - Version Control

- Use version control software to organize code
- Compare and contrast different strategies for using version control
- Choose a suitable version control strategy for a team and project

Testing and Maintenance

- Explain why code should be tested, with concrete examples
 - List and describe the levels of testing
 - Explain the purpose of test-driven development
-

Outside-of-class work

Readings

- A Beginner's Guide to Git Version Control
- Collaborating with Merge Requests in GitLab
- The Testing Pyramid
- Test-Driven Development (TDD): A Step-by-Step Guide

Optional:

- Git Branching and Merging - A Visual Guide
 - TDD: Where Did It All Go Wrong? (YouTube)
 - GitLab Flow vs Git Flow vs Trunk-Based Development
 - Writing Better Unit Tests
-

Pacing and sequencing notes (important)

This week benefits from **activity-flexible time**, especially for hands-on Git and testing exercises. Some activities can be shortened or demonstrated if time is constrained.

- Students may meet **additional group members** partway through the week
- Teams may still be stabilizing; activities are designed to support communication and shared troubleshooting

Instructors should expect to run **2-4 activities total**, depending on time and tooling availability.

Instructor notes

Students often:

- Treat Git as a set of commands rather than a collaboration tool
- Fear merge conflicts instead of viewing them as normal
- Think of testing as something done “after” code is written

Normalize confusion and mistakes. Struggle is expected and productive this week.

Activity 1 - Git Team Scavenger Hunt

Learning focus: Git workflows, collaboration, confidence

Purpose

Give every student hands-on experience with common version control operations in a low-stakes, supported environment.

Learning objectives targeted

- Use version control software to organize code
- Compare different strategies for using version control

Team focus

Peer support, communication

Recommended group size

Individuals working within teams

Time estimate

30–45 minutes

Best for

Activity-flexible time

Description

Students complete a structured list of Git tasks (e.g., clone a repo, create a branch, commit changes, open a merge request, resolve a conflict). Repositories are designed to trigger simple conflicts. Teammates are encouraged to help each other.

Completion can be tracked via screenshots or short logs.

Debrief prompts

- Which step felt most confusing?
- How did teammates help when someone got stuck?
- What practices reduced friction?

Notes / Reflections

Activity 2 - Branching Model Simulation (Human Git Tree)

Learning focus: branching, merging, conflicts

Purpose

Make abstract Git concepts concrete through physical simulation.

Learning objectives targeted

- Compare different strategies for using version control

Team focus

Shared understanding, communication

Recommended group size

Whole class or large groups

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Students physically represent commits and branches using space, tape, or yarn. The instructor narrates scenarios involving branching, merging, and conflicts. One student resolves a simulated conflict.

Debrief prompts

- What caused conflicts in this simulation?
- How does frequent integration help?
- Which Git practices map to these actions?

Notes / Reflections

Activity 3 - Merge Conflict Race

Learning focus: conflict resolution, teamwork

Purpose

Reduce anxiety around merge conflicts by turning them into a collaborative challenge.

Learning objectives targeted

- Use version control software to organize code
- Choose a suitable strategy for version control

Team focus

Coordination, problem-solving

Recommended group size

Pairs or triads

Time estimate

25–35 minutes

Best for

Activity-flexible time

Description

Teams intentionally trigger and resolve a merge conflict in a prepared repository. The first team to resolve the conflict cleanly wins, but all teams must complete the task.

Debrief prompts

- What caused the conflict?
- How did you decide what to keep?
- What would prevent this conflict in real work?

Notes / Reflections

Activity 4 - Testing Pyramid Quiz & Discussion

Learning focus: testing levels, purpose

Purpose

Introduce testing levels and why they are structured hierarchically.

Learning objectives targeted

- Explain why code should be tested
- List the levels of testing

Team focus

Conceptual alignment

Recommended group size

Any

Time estimate

20–30 minutes

Best for

Lecture-constrained time

Description

Students predict the shape of the testing pyramid, then discuss examples of unit, integration, and end-to-end tests for a simple system.

Debrief prompts

- Why are unit tests emphasized?
- What risks come with relying on only end-to-end tests?

Notes / Reflections

Activity 5 - Test Case Design Relay

Learning focus: thinking beyond the happy path

Purpose

Practice designing thorough unit test cases.

Learning objectives targeted

- Explain why code should be tested

Team focus

Shared reasoning

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams take turns writing test cases for a simple function, focusing on edge cases and invalid inputs.

Debrief prompts

- Which cases were easiest to miss?
- How did others' ideas expand your thinking?

Notes / Reflections

[illegible]

Activity 6 - TDD Ping-Pong Pairing

Learning focus: test-first thinking

Purpose

Experience test-driven development as a collaborative process.

Learning objectives targeted

- Explain the purpose of test-driven development

Team focus

Communication, coordination

Recommended group size

Pairs

Time estimate

20–30 minutes

Best for

Activity-flexible time

Description

Pairs alternate writing tests and implementation for a small problem. Optionally rotate partners mid-way.

Debrief prompts

- How did writing tests first affect your approach?
- What felt slower or faster than expected?
- When might TDD be most useful?

Notes / Reflections

Week 4 - Exceptional Testing & Technical Debt

Week overview

This week deepens students' understanding of **testing beyond the happy path** and introduces **technical debt** as a practical and ethical concern in real projects. Students examine how exceptional cases, dependencies, and shortcuts affect software quality over time, and how testing strategies and refactoring decisions interact with project velocity.

This week also sits at a critical point in project execution. Teams are actively building, submitting their first iteration, and responding to feedback. Class time intentionally supports **reflection, decision-making, and responsibility-sharing**, alongside technical skill development.

Learning objectives

By the end of this week, students should be able to:

Testing and Maintenance

- Compare and contrast strategies for approaching testing
- Explain the purpose of test-driven development
- Test exceptional and error-handling code
- Use a framework to write automated unit tests
- Explain how errors and exceptions should be handled

Project Execution

- Define and identify technical debt
- Describe the Agile Software Development process
- Explain how up-front planning artifacts fit into Agile execution
- Assign responsibilities to project stakeholders

Outside-of-class work

Readings

- Differentiating Test Doubles
- Software testing lessons from edge cases

- What is Technical Debt? (YouTube)
- A Few Things I've Learned About Agile

Optional:

- What is an Edge Case in Software Testing?
 - The 4 Main Agile Roles
 - Agile Velocity vs Capacity
-

Deliverables

- **Iteration 0 Due:** Monday, January 26 @ 4:30 PM
 - **Iteration 0 Peer Review Due:** Wednesday, January 28 @ 4:30 PM
-

Pacing and sequencing notes (important)

This week includes **lecture-constrained time** to review deliverables and expectations at the start of the week.

Instructors should plan to:

- Briefly review Iteration 0 and peer review expectations
- Prioritize activities that connect testing and technical debt to *current project work*

Not all activities are expected to be run. Select based on time, energy, and team needs.

Instructor notes

Students often:

- Test only “normal” behavior and overlook failure paths
- View technical debt as purely technical rather than organizational
- Struggle to balance short-term progress with long-term quality

This week is an opportunity to normalize trade-offs, mistakes, and course correction as part of Agile practice.

Activity 1 - Edge Case Brainstorm (Bug Hunt)

Learning focus: exceptional paths, testing creativity

Purpose

Practice identifying edge cases and failure scenarios that require explicit testing.

Learning objectives targeted

- Test exceptional and error-handling code
- Explain why code should be tested

Team focus

Creative thinking, shared responsibility

Recommended group size

3–5 students

Time estimate

15–25 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Given a function or feature specification with hidden pitfalls, teams brainstorm as many edge cases and failure scenarios as possible. Unique edge cases earn points.

Introduce test doubles conceptually by asking how external dependencies could be controlled during testing.

Debrief prompts

- Which edge cases were easiest to miss?
- How does this change how you think about testing?

Notes / Reflections

Activity 2 - Mock Objects Role-Play

Learning focus: test doubles, controlled dependencies

Purpose

Illustrate how stubs and mocks support testing of exceptional behavior.

Learning objectives targeted

- Compare strategies for approaching testing
- Test exceptional code paths

Team focus

Conceptual understanding, communication

Recommended group size

Whole class or small groups

Time estimate

15–20 minutes

Best for

Lecture-constrained time

Description

Students role-play a system under test and its dependencies. Dependencies are replaced with controlled behaviors (fixed responses or errors) to demonstrate how test doubles work.

Debrief prompts

- Why would we replace real components in tests?
- What kinds of behavior become testable this way?

Notes / Reflections

Activity 3 - Technical Debt Jenga

Learning focus: technical debt, system fragility

Purpose

Visually demonstrate how shortcuts and neglect accumulate into instability.

Learning objectives targeted

- Define and identify technical debt
- Explain long-term impacts of design decisions

Team focus

Systems thinking, reflection

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Activity-flexible time

Description

Teams build and modify a tower under increasing time pressure, simulating rushed changes and ignored refactoring. Collapsing towers serve as a metaphor for accumulating technical debt.

Debrief prompts

- What actions made the structure unstable?
- How does this map to real project behavior?

Notes / Reflections

Activity 4 - Technical Debt “Investment” Discussion

Learning focus: trade-offs, prioritization

Purpose

Practice reasoning about when to pay down technical debt versus shipping features.

Learning objectives targeted

- Define and identify technical debt
- Assign responsibilities to project stakeholders

Team focus

Decision-making, justification

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams debate a scenario where time must be allocated either to new features or refactoring. Each group presents and justifies their decision.

Debrief prompts

- What risks come with delaying refactoring?
- Who bears the cost of technical debt?

Notes / Reflections

Activity 5 - Consensus Conflict Resolution Exercise

Learning focus: conflict management, shared responsibility

Purpose

Practice constructive approaches to common team conflicts.

Learning objectives targeted

- Assign responsibilities to project stakeholders
- Describe Agile team collaboration

Team focus

Communication, empathy, consensus

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Activity-flexible time

Description

Teams are given a realistic conflict scenario and must reach consensus on a response plan, ensuring all voices are heard.

Debrief prompts

- What made consensus difficult?
- What strategies helped move forward?

Notes / Reflections

Activity 6 - Five Whys Drill-Down

Learning focus: root cause analysis, continuous improvement

Purpose

Introduce a structured technique for diagnosing underlying causes of problems.

Learning objectives targeted

- Describe Agile continuous improvement practices
- Explain how issues relate to process and responsibility

Team focus

Reflection, analytical thinking

Recommended group size

3–5 students

Time estimate

10–20 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams apply the Five Whys technique to a technical or team issue (real or hypothetical) to uncover root causes beyond surface symptoms.

Debrief prompts

- What surprised you about the root cause?
- How could this insight change future behavior?

Notes / Reflections

[illegible]

Week 5 - Design Principles & Refactoring

Week overview

This week focuses on **software design quality and continuous improvement**. Students learn to recognize poor design through code smells, understand how design principles shape coupling and cohesion, and practice refactoring code while preserving behavior.

Design principles are framed not as abstract rules, but as **tools for reasoning about change**. Class time emphasizes reading, critiquing, and improving existing code-skills that are essential for maintaining real systems and managing technical debt over time.

Learning objectives

By the end of this week, students should be able to:

Software Design

- List and define core software design principles
 - Explain how design principles affect coupling and cohesion
 - Identify code that violates SOLID design principles
 - Recognize common code smells
 - Refactor code to improve structure without changing behavior
-

Outside-of-class work

Readings

- SOLID: The First 5 Principles of Object-Oriented Design
- Code Smells
- Clean Code
- Mastering SOLID Principles, Design Patterns, and Refactoring
- Clean Code and Coding Best Practices

Optional:

- A Solid Guide to SOLID Principles
- How to Refactor
- Code Refactoring: Keeping Your Codebase Healthy

- Code Smells and Refactoring - Handbook of Software Engineering
-

Pacing and sequencing notes (important)

This week works best with **activity-flexible time**, as students benefit from sustained engagement with code examples.

Instructors should expect to:

- Spend minimal time introducing terminology
 - Spend most class time on **reading, discussing, and modifying code**
 - Emphasize that refactoring is ongoing and incremental, not a one-time cleanup
-

Instructor notes

Students often:

- Believe clean code is subjective or stylistic
- Focus on syntax rather than structure
- Hesitate to refactor working code

Reinforce that design principles exist to **support change, comprehension, and collaboration**, not to enforce rigid rules.

Activity 1 - Code Smells Scavenger Hunt Session

Learning focus: identifying poor design, critical reading of code

Purpose

Train students to quickly recognize common code smells and design problems.

Learning objectives targeted

- Identify code that violates SOLID principles
- Recognize common code smells

Team focus

Shared analysis, discussion

ReW5 - group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams analyze a prepared “smelly” Java code snippet and identify as many design issues as possible (e.g., long methods, duplicate code, magic numbers, unclear naming). Each valid smell earns a point.

The instructor compiles a master list and briefly connects smells to violated principles (e.g., SRP, low cohesion).

Debrief prompts

- Which smells were easiest to spot?
- Which required deeper reasoning?
- How do smells signal future maintenance problems?

Notes / Reflections

Activity 2 - SOLID Design Poker Session

Learning focus: applying design principles to concrete scenarios

Purpose

Practice mapping real design decisions to SOLID principles.

Learning objectives targeted

- List and define software design principles
- Identify violations of SOLID principles

Team focus

Interpretation, justification

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Each team receives cards labeled S, O, L, I, D. The instructor reads design scenarios, and teams select the principle most relevant to the scenario (violated or improved). Teams justify their choices.

Debrief prompts

- Why did different teams choose different principles?
- Can multiple principles apply at once?
- How do principles interact or conflict?

Notes / Reflections

Activity 3 - Refactoring Golf

Learning focus: incremental refactoring, cohesion and coupling

Purpose

Practice improving code structure with minimal, intentional changes.

Learning objectives targeted

- Refactor code without changing behavior
- Explain how design principles affect coupling and cohesion

Team focus

Strategic thinking, collaboration

Recommended group size

3–5 students

Time estimate

30–40 minutes

Best for

Activity-flexible time

Description

Teams refactor a working but poorly designed code snippet. Each structural improvement counts as a “stroke.” The goal is to meaningfully improve the design in as few strokes as possible.

Teams explain their refactoring choices afterward.

Debrief prompts

- Which refactors gave the biggest improvement?
- What did you choose *not* to refactor, and why?
- How does this mirror real project constraints?

Notes / Reflections

Activity 4 - Mob Programming Refactor Session

Learning focus: collaborative refactoring, collective code ownership

Purpose

Demonstrate how teams can refactor code collaboratively and share design reasoning.

Learning objectives targeted

- Identify and refactor code that violates design principles
- Explain design decisions clearly to others

Team focus

Communication, shared ownership

Recommended group size

Whole class

Time estimate

30–45 minutes

Best for

Activity-flexible time

Description

The class refactors a problematic code snippet together using a mob programming format. One student acts as the driver while others guide refactoring decisions. Roles may rotate.

Debrief prompts

- How were disagreements resolved?
- What did others notice that you missed?
- How does collective ownership affect code quality?

Notes / Reflections

Activity 5 - AI Pair-Programming Challenge

Learning focus: critical evaluation of AI-generated code

Purpose

Develop judgment when using AI tools for design and refactoring.

Learning objectives targeted

- Identify design issues and code smells
- Apply clean code principles critically

Team focus

Evaluation, ethical tool use

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams prompt an AI tool to refactor or improve a code snippet. They then evaluate the result, identifying improvements, violations, or new smells introduced by the AI.

Each team reports one useful suggestion and one problem.

Debrief prompts

- What did the AI do well?
- What required human judgment?
- How should teams use AI responsibly in refactoring?

Notes / Reflections

Week 6 - Alternatives to Inheritance & Dependency Injection

Week overview

This week focuses on **design flexibility and decoupling**, emphasizing alternatives to inheritance and techniques that support change, testability, and error isolation. Students examine the limitations of deep inheritance hierarchies and learn how composition, strategy, and dependency injection can reduce coupling and improve maintainability.

Because this week includes a **term test** and is immediately followed by **Reading Week**, in-class time is constrained. Class activities prioritize conceptual understanding, visualization, and refactoring patterns that students can continue practicing independently.

Learning objectives

By the end of this week, students should be able to:

Software Design

- Explain the limitations of over-applying inheritance
 - List and define alternatives to inheritance
 - Refactor existing code to better adhere to SOLID design principles
 - Use composition and dependency injection to reduce coupling
 - Explain appropriate strategies for error handling and propagation
-

Outside-of-class work

Readings

- Composition over Inheritance
- Strategy Design Pattern in Java - Example Tutorial
- Strategy in Java / Design Patterns
- Composition over Inheritance: Lessons Learned

Optional:

- Strategy Design Pattern: Write Flexible Code Without Conditionals
- Composition over Inheritance (Veerpal Brar)

- Strategy Design Pattern and Open Closed Principle in Java
 - Composition over Inheritance - Game Development Example
-

Pacing and sequencing notes (important)

This week includes a **term test** at the end of the week, which significantly reduces activity-flexible class time.

- In-class activities should be selected sparingly
 - Priority should be given to **conceptual clarity** and **pattern recognition**
 - Additional practice is intentionally provided outside of class, especially given the upcoming Reading Week
-

Instructor notes

Students often:

- Default to inheritance for reuse, even when behavior varies
- Conflate polymorphism with inheritance
- Catch exceptions too broadly or too early
- Struggle to see how design choices affect testing

Emphasize that this week is about **design decision-making**, not memorizing patterns.

Activity 1 - Inheritance vs. Composition Debate

Learning focus: design trade-offs, flexibility

Purpose

Encourage critical thinking about when inheritance is helpful and when it becomes limiting.

Learning objectives targeted

- Explain limitations of inheritance
- List alternatives to inheritance

Team focus

Reasoning, justification

Recommended group size

Whole class or large groups

Time estimate

20–25 minutes

Best for

Lecture-constrained time

Description

Students argue for inheritance or composition in a given design scenario, then reflect on how composition often enables easier extension and fewer unintended constraints.

Debrief prompts

- What problems emerged with inheritance?
- What kinds of change does composition support better?

Notes / Reflections

Activity 2 - Strategy Pattern Role-Play

Learning focus: composition, runtime behavior

Purpose

Make the Strategy pattern concrete and memorable.

Learning objectives targeted

- List and define alternatives to inheritance

Team focus

Conceptual understanding, communication

Recommended group size

Small group or whole class

Time estimate

15–20 minutes

Best for

Lecture-constrained time

Description

Students act out a Context and interchangeable Strategies to demonstrate runtime behavior changes without modifying the Context.

Debrief prompts

- How does this differ from subclassing?
- Why is this more flexible?

Notes / Reflections

[illegible]

Activity 3 - Refactor Inheritance to Composition Exercise

Learning focus: refactoring for flexibility

Purpose

Practice replacing inheritance hierarchies with composition.

Learning objectives targeted

- Refactor code to adhere to SOLID principles

Team focus

Design reasoning, collaboration

Recommended group size

3–5 students

Time estimate

25–35 minutes

Best for

Activity-flexible time

Description

Teams redesign a problematic inheritance hierarchy using composition and strategy, producing UML sketches or pseudocode.

Debrief prompts

- What dependencies were removed?
- What became easier to change?

Notes / Reflections

[illegible]

Activity 4 - Dependency Injection Kata

Learning focus: decoupling, testability

Purpose

Show how dependency injection reduces coupling and enables testing.

Learning objectives targeted

- Refactor existing code to reduce coupling

Team focus

Problem-solving, collaboration

Recommended group size

Pairs

Time estimate

15–25 minutes

Best for

Activity-flexible time

Description

Students refactor a tightly coupled class so dependencies are injected rather than constructed internally.

Debrief prompts

- What changed about testability?
- How does this support SOLID design?

Notes / Reflections

[illegible]

Activity 5 - Error-Handling Hot Potato

Learning focus: exception propagation

Purpose

Visualize how errors propagate and where they should be handled.

Learning objectives targeted

- Explain how errors and exceptions should be handled

Team focus

Shared understanding

Recommended group size

Whole class

Time estimate

15–20 minutes

Best for

Lecture-constrained time

Description

Students physically pass an “exception” through a call chain, deciding whether to handle or propagate it.

Debrief prompts

- Where was handling appropriate?
- What happens when errors are swallowed?

Notes / Reflections

[illegible]

Activity 6 - Exception Testing Challenge

Learning focus: testing exceptional paths

Purpose

Practice writing tests that validate error behavior.

Learning objectives targeted

- Test exceptional code
- Use unit testing frameworks appropriately

Team focus

Precision, correctness

Recommended group size

Pairs or small groups

Time estimate

15–20 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams write a unit test that asserts a specific exception is thrown under invalid conditions.

Debrief prompts

- What makes a good exception test?
- How does DI help here?

Notes / Reflections

[illegible]

Activity 7 - Midterm Retrospective (Team Check-In)

Learning focus: reflection, continuous improvement

Purpose

Pause to reflect on team practices at the course midpoint.

Learning objectives targeted

- Apply Agile reflection practices
- Assign and adjust team responsibilities

Team focus

Communication, accountability

Recommended group size

Permanent project teams

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams complete a Start / Stop / Continue retrospective focused on project execution and collaboration.

Debrief prompts

- What is helping the team most?
- What needs to change going forward?

Notes / Reflections

[illegible]

Suggested at-home practice activities

Notes / Reflections

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Week 7 - Design Patterns & Architecture

Week overview

This week focuses on **design patterns and architectural styles** as tools for structuring software systems and guiding refactoring decisions. Students learn to recognize common creational, structural, and behavioral patterns, understand when they are appropriate, and apply them deliberately rather than mechanically.

Architecture is framed as a set of **trade-offs and communication decisions**, not a single “correct” structure. Class activities emphasize identifying patterns in context, justifying design choices, and connecting lower-level patterns to higher-level architectural styles such as MVC and layered architectures.

Learning objectives

By the end of this week, students should be able to:

Software Design

- List and define major categories of design patterns (creational, structural, behavioral)
 - Identify a suitable design pattern for solving a design problem
 - Identify a suitable design pattern for refactoring existing code
 - Explain how design patterns support flexibility, extensibility, and maintainability
 - Relate design patterns to architectural styles
-

Outside-of-class work

Core references (Refactoring Guru)

Students are encouraged to use the following reference material throughout the week:

- What is a Design Pattern?
- Classification of Patterns
- Builder
- Singleton
- Abstract Factory
- Adapter

- Facade
- Special Case
- Chain of Responsibility
- Observer
- Iterator

Readings

- Exploring the Factory Method Design Pattern
- Adapter Pattern - What It Is and How to Use It?
- Observer Design Pattern: A Complete Guide with Examples
- MVC Architecture
- Understanding Layered Architecture

Optional:

- Factory Design Pattern in Java
- Adapter Design Pattern
- Observer Design Pattern
- Layered Architecture: Still a Solid Approach

Pacing and sequencing notes (important)

This week benefits from **activity-flexible time**, as students need space to reason, discuss, and justify design choices.

Instructors should:

- Avoid attempting to “cover” every pattern exhaustively
 - Emphasize **recognition, justification, and trade-offs**
 - Reinforce that patterns are tools, not goals
-

Instructor notes

Students often:

- Try to memorize patterns rather than understand when to use them
- Apply patterns too early or unnecessarily

- Confuse architectural styles with design patterns

Reinforce that **clarity and appropriateness matter more than pattern count.**

Activity 1 - Pattern Charades

Learning focus: pattern recognition, roles, behavior

Purpose

Reinforce understanding of pattern intent and participant roles in a memorable way.

Learning objectives targeted

- List and define different types of design patterns

Team focus

Shared understanding, communication

Recommended group size

Small teams or whole class

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams act out or diagram a design pattern without naming it. Other teams guess the pattern and explain their reasoning.

Debrief prompts

- Which aspects of the pattern were easiest to convey?
- What roles were essential to include?

Notes / Reflections

[illegible]

Activity 2 - Real-World Pattern Hunt

Learning focus: applying patterns to problems

Purpose

Practice selecting appropriate patterns for concrete scenarios.

Learning objectives targeted

- Identify a suitable design pattern for solving a problem

Team focus

Reasoning, justification

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams map real-world scenarios or system descriptions to suitable design patterns and justify their choices. Multiple valid answers are discussed.

Debrief prompts

- Why did this pattern fit the problem?
- What alternatives were possible?

Notes / Reflections

[illegible]

Activity 3 - Architecture Kata (Mini)

Learning focus: architecture design, pattern integration

Purpose

Practice designing a system architecture and identifying where patterns fit.

Learning objectives targeted

- Identify design patterns within architectural designs
- Relate patterns to architectural styles

Team focus

Design thinking, communication

Recommended group size

3–5 students

Time estimate

30–40 minutes

Best for

Activity-flexible time

Description

Teams design a high-level architecture for a given system, identifying layers and points where design patterns apply. Peer reviewers rotate to ask questions and suggest improvements.

Debrief prompts

- Why did you choose this structure?
- How do patterns support your architecture?

Notes / Reflections

[illegible]

Activity 4 - MVC Role-Play

Learning focus: separation of concerns, flow of control

Purpose

Make the MVC architectural pattern concrete and intuitive.

Learning objectives targeted

- List and define architectural design patterns
- Explain separation of concerns

Team focus

Conceptual understanding

Recommended group size

Small groups or whole class

Time estimate

15–20 minutes

Best for

Lecture-constrained time

Description

Students role-play Model, View, and Controller responding to user input. Variations can show multiple views or updates.

Debrief prompts

- Why does MVC reduce coupling?
- What changes become easier under this structure?

Notes / Reflections

[illegible]

Activity 5 - Design Pattern Show and Tell

Learning focus: teaching, synthesis

Purpose

Deepen understanding by explaining a pattern to others.

Learning objectives targeted

- List and define design patterns
- Identify appropriate use cases

Team focus

Communication, synthesis

Recommended group size

3–5 students

Time estimate

30–40 minutes

Best for

Activity-flexible time

Description

Each team prepares a short teach-back on an assigned pattern, including a diagram and real-world analogy.

Debrief prompts

- What made the pattern easy or hard to explain?
- What misconceptions came up?

Notes / Reflections

[illegible]

Activity 6 - Pattern-Based Code Refactoring

Learning focus: refactoring with intent

Purpose

Apply patterns to improve existing code.

Learning objectives targeted

- Identify a suitable design pattern for refactoring existing code

Team focus

Problem-solving, justification

Recommended group size

3–5 students

Time estimate

30–40 minutes

Best for

Activity-flexible time

Description

Teams refactor a provided code snippet using a suitable design pattern (e.g., Factory, Strategy).

Teams explain how the refactor improved extensibility or readability.

Debrief prompts

- What problem did the pattern solve?
- How did it change the code's flexibility?

Notes / Reflections

[illegible]

Activity 7 - Architecture Comparison Discussion

Learning focus: architectural trade-offs

Purpose

Encourage comparison of architectural styles at a high level.

Learning objectives targeted

- List and define architectural design patterns
- Explain trade-offs between architectures

Team focus

Analytical thinking

Recommended group size

Pairs or triads

Time estimate

15–25 minutes

Best for

Lecture-constrained time

Description

Students compare two architectural styles (e.g., MVC vs layered) and identify advantages, disadvantages, and appropriate contexts.

Debrief prompts

- What problems does each style solve well?
- What kinds of systems would struggle under it?

Notes / Reflections

[illegible]

Week 8 - Advanced Testing

Week overview

This week builds students' testing skills beyond basic unit tests, focusing on **test doubles (stubs and mocks)** and on choosing the right test type for the job (unit vs. integration vs. end-to-end).

The emphasis is on isolating behavior under test, controlling dependencies, and understanding what different testing layers do (and do not) guarantee.

This week also connects testing practice to broader engineering concerns: complex systems fail at boundaries, and good tests help teams detect integration problems early, reduce uncertainty, and maintain confidence as the codebase evolves.

Learning objectives

By the end of this week, students should be able to:

Testing and Maintenance

- Explain the purpose of stubs and mocks in testing
 - Compare and contrast stubs and mocks
 - Write isolated unit tests with mock dependencies
 - Explain the differences between unit tests, integration tests, and end-to-end tests
 - Write integration tests and end-to-end tests
-

Outside-of-class work

Readings

- Understanding the “ilities” of Software Architecture
- The Cost of Code
- Scalability vs. Performance: What's the Difference?
- Avoiding YAGNI Doesn't Mean You Shouldn't Plan

Optional:

- Why Readable Code Matters More Than Clever Code
- Maintainability: A Key Aspect of Software Quality

- Scalability Design Principles
 - Design for Change - A Key Software Engineering Principle
-

Pacing and sequencing notes (important)

This week benefits from **activity-flexible time**, especially for activities involving running tests, debugging integration failures, or using CI tooling.

If tooling or environment setup is a constraint, prioritize activities that focus on **test design decisions and reasoning** over implementation.

Instructors should expect to run **2-4 activities total**, depending on available infrastructure.

Instructor notes

Students often:

- Use mocks and stubs interchangeably without clear intent
- Write tests that accidentally depend on time, randomness, or external services
- Confuse “a lot of tests” with “the right tests”
- Over-rely on end-to-end tests because they feel most realistic

Reinforce that the goal is not maximal testing, but **appropriate testing** that provides fast feedback and isolates failures.

Activity 1 - Continuous Integration Pipeline Race

Learning focus: integration failures, CI feedback loops

Purpose

Practice diagnosing and fixing integration issues using CI feedback.

Learning objectives targeted

- Write integration tests
- Explain differences between test levels

Team focus

Coordination, problem-solving

Recommended group size

Pairs or small teams

Time estimate

30–45 minutes

Best for

Activity-flexible time

Description

Teams start with a small project where an integration test fails due to a mismatch between components. Teams fix the issue, push changes, and use CI output to verify success. The first team to pass CI “wins,” but all teams complete the exercise.

Debrief prompts

- What caused the integration failure?
- What evidence helped you find it fastest?
- What would you add to prevent regression?

Notes / Reflections

Activity 2 - Microservice Integration Simulation

Learning focus: integration points, contracts, end-to-end flow

Purpose

Reason about integration testing without requiring full implementation.

Learning objectives targeted

- Explain differences between unit, integration, and end-to-end tests
- Write integration and end-to-end test plans (conceptually)

Team focus

Systems thinking, communication

Recommended group size

3–5 students

Time estimate

25–35 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams receive “service cards” describing simple APIs (inputs/outputs). They design an end-to-end flow for a user scenario and identify what to verify at each integration boundary. Introduce a twist (format mismatch, timeout, delayed response) and have teams adjust.

Debrief prompts

- Where are the integration risks?
- What would you validate with unit tests vs. integration tests vs. end-to-end tests?

Notes / Reflections

Activity 3 - Test Doubles Showdown

Learning focus: choosing stubs vs. mocks, test intent

Purpose

Clarify when to use stubs, mocks, fakes, and real dependencies.

Learning objectives targeted

- Explain the purpose of stubs and mocks
- Compare and contrast stubs and mocks

Team focus

Justification, precision

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams are given a scenario with multiple dependencies (e.g., database + payment service). For each dependency, they choose an appropriate double (stub/mock/fake/real) and explain why. Optionally follow with a brief demo of how a stub vs. mock is implemented in their testing framework.

Debrief prompts

- What are you trying to prove with this test?
- When is verifying “called correctly” important?
- When is returning controlled data enough?

Notes / Reflections

Activity 4 - Flaky Test Roulette

Learning focus: nondeterminism, test stability

Purpose

Make flaky tests tangible and discuss strategies to eliminate nondeterminism.

Learning objectives targeted

- Compare strategies for approaching testing
- Write reliable tests using controlled dependencies

Team focus

Diagnostic thinking

Recommended group size

Any

Time estimate

15–25 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Students run a test that fails intermittently due to randomness or timing. Teams propose and implement fixes (seed control, time abstraction, dependency mocking, tolerance adjustments).

Debrief prompts

- What made the test flaky?
- What change made it deterministic?
- What kinds of real systems cause flakiness?

Notes / Reflections

Activity 5 - Acceptance Criteria Workshop

Learning focus: behavior specification, end-to-end thinking

Purpose

Practice writing acceptance criteria as testable behavioral statements.

Learning objectives targeted

- Write end-to-end tests (in planned form)
- Explain differences between test levels

Team focus

Clarity, shared understanding

Recommended group size

3–5 students

Time estimate

25–35 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams take a user story (preferably from their project) and write acceptance criteria in plain language or a structured form (e.g., Given/When/Then). Teams exchange and peer review for clarity and completeness.

Debrief prompts

- Where was ambiguity hiding?
- What acceptance criteria were missing at first?
- How do acceptance tests complement unit tests?

Notes / Reflections

Activity 6 - Bug Triage Meeting (Role-Play)

Learning focus: prioritization, quality strategy

Purpose

Practice prioritizing defects and identifying which issues should drive new tests.

Learning objectives targeted

- Compare strategies for approaching testing
- Explain how test strategy supports maintenance

Team focus

Decision-making, stakeholder thinking

Recommended group size

3–6 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams receive a set of bug reports and prioritize them from multiple perspectives (user impact, reproducibility, severity, risk). Teams decide which bugs should lead to new tests and what level of test is appropriate.

Debrief prompts

- What made a bug urgent vs. deferrable?
- Which bugs indicate missing unit tests vs missing integration tests?
- What would you automate?

Notes / Reflections

Activity 7 - Continuous Testing with AI Pairing

Learning focus: critical evaluation of AI-generated tests

Purpose

Practice using AI tools to draft tests while applying human judgment for correctness and stability.

Learning objectives targeted

- Write integration or API-level tests
- Compare strengths/limits of automation approaches

Team focus

Critical evaluation, responsible tool use

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Activity-flexible time

Description

Teams prompt an AI tool to generate an integration test or API test, then evaluate and revise it. Focus on whether the test is meaningful, stable, and aligned with acceptance criteria.

Debrief prompts

- What did the AI capture well?
- What context did it miss?
- What risks could make the test flaky or misleading?

Notes / Reflections

Week 9 - Legacy Software, Architecture Recovery & Change

Week overview

This week focuses on **legacy software**: systems that still deliver value but are difficult to understand, test, or change. Students learn strategies for making sense of unfamiliar codebases, recovering architecture with little or no documentation, and making **safe, incremental changes** to fragile systems.

Legacy software is framed as the *norm*, not the exception. Most real-world development involves understanding existing systems before extending them. Class activities emphasize reading code, inferring intent, writing characterization tests, and minimizing risk when adding features.

Learning objectives

By the end of this week, students should be able to:

Testing and Maintenance

- Build a legacy software system from source
 - Identify the purpose of a legacy software system
 - Describe the architecture of a legacy system with little or no documentation
 - Write tests for a legacy software system
 - Add a new feature to a legacy software system safely
-

Outside-of-class work

Readings

- How Morgan Stanley Tackled One of Coding's Toughest Problems
- Notes from *Working Effectively with Legacy Code*
- Working with Legacy Code: 9 Life-Saving Strategies
- Mazda Powers Legacy Migration with OutSystems

Optional:

- Case Study Challenge - Small Agency Legacy Application Migration

- Working Well With Legacy Code
 - Legacy System Migration: Finding Your Risk-Free Approach
 - COBOL to Java and Newspapers Still Get Delivered
-

Pacing and sequencing notes (important)

This week benefits from **activity-flexible time**, especially for reading and reasoning about code. Some activities may be run without compiling or executing software if setup time is a constraint.

Instructors should:

- Emphasize *understanding before changing*
 - Normalize uncertainty and slow progress
 - Encourage students to verbalize what they know versus what they are inferring
-

Instructor notes

Students often:

- Assume legacy code is “bad” rather than “contextual”
- Want to refactor immediately without understanding behavior
- Underestimate how much knowledge is embedded in tests and usage patterns

Reinforce that legacy work rewards **patience, curiosity, and restraint**.

Activity 1 - “Swap an Old Assignment” Legacy Exercise

Learning focus: understanding undocumented systems

Purpose

Experience the challenge of interpreting unfamiliar code written without the benefit of documentation or context.

Learning objectives targeted

- Identify the purpose of a legacy system
- Describe system behavior with limited information

Team focus

Empathy, communication, interpretation

Recommended group size

Pairs or triads

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Students bring a programming assignment from a previous course (or are given one by the instructor). They swap assignments with a partner, who must determine:

- What the program is supposed to do
- How it works at a high level
- Where they would start if asked to modify it

Original authors may clarify afterward.

Debrief prompts

- What information was hardest to infer?
- What assumptions did you make?
- How did missing context affect confidence?

Notes / Reflections

Activity 2 - Legacy Code Scavenger Hunt

Learning focus: navigating unfamiliar code

Purpose

Practice locating functionality and structure in a messy or undocumented codebase.

Learning objectives targeted

- Describe the architecture of a legacy system
- Build confidence reading unfamiliar code

Team focus

Problem-solving, persistence

Recommended group size

Individuals or pairs

Time estimate

25–35 minutes

Best for

Activity-flexible time

Description

Students answer specific questions about a provided legacy codebase (e.g., where logic lives, which variables control behavior). Answers require reading and tracing code, not guessing.

Debrief prompts

- What strategies helped you navigate the code?
- What tools or notes would have helped?

Notes / Reflections

[illegible]

Activity 3 - Strangler Fig Refactoring Plan

Learning focus: safe, incremental change

Purpose

Practice planning refactoring without rewriting entire systems.

Learning objectives targeted

- Add a new feature to a legacy system safely
- Write tests to protect existing behavior

Team focus

Planning, risk management

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams receive a legacy function or module and propose a step-by-step plan to refactor or extend it safely, emphasizing tests and small changes.

Debrief prompts

- What would you test first?
- Where is the highest risk?
- What change would you delay?

Notes / Reflections

Activity 4 - “Telephone” Code Communication

Learning focus: documentation loss, ambiguity

Purpose

Demonstrate how unclear communication leads to misunderstood requirements.

Learning objectives targeted

- Identify the purpose of a system
- Explain why documentation matters in legacy code

Team focus

Listening, interpretation

Recommended group size

Small groups

Time estimate

15–20 minutes

Best for

Lecture-constrained time

Description

A requirement is passed through multiple students before becoming a test or usage example. The final result is compared to the original.

Debrief prompts

- Where did meaning change?
- How does this mirror legacy systems?

Notes / Reflections

Activity 5 - Legacy Refactoring Golf (Coding Dojo)

Learning focus: refactoring under constraints

Purpose

Practice improving testability without changing behavior.

Learning objectives targeted

- Write tests for legacy code
- Prepare code for safe modification

Team focus

Technique, restraint

Recommended group size

Small groups or whole class

Time estimate

30–45 minutes

Best for

Activity-flexible time

Description

Students work with a small legacy function that is difficult to test. The goal is to introduce seams or abstractions that enable testing, with minimal changes.

Debrief prompts

- What made this code hard to test?
- What was the smallest helpful change?

Notes / Reflections

[illegible]

Activity 6 - Write the Missing Documentation

Learning focus: explanation and clarity

Purpose

Practice articulating what code does in plain language.

Learning objectives targeted

- Describe system behavior clearly
- Prepare systems for future maintainers

Team focus

Clarity, precision

Recommended group size

Individuals or pairs

Time estimate

15–20 minutes

Best for

Lecture-constrained time

Description

Students write a short docstring or explanation for a confusing legacy function.

Debrief prompts

- What parts were hardest to explain?
- What would you change to make this clearer?

Notes / Reflections

[illegible]

Activity 7 - Regression Testing Strategy Challenge

Learning focus: protecting behavior

Purpose

Develop a practical checklist for adding features safely.

Learning objectives targeted

- Write tests for legacy systems
- Add features without breaking existing behavior

Team focus

Strategy, risk awareness

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams design a testing and validation strategy for adding a new feature to a system with no tests.

Debrief prompts

- What would you test first?
- What behavior must not change?

Notes / Reflections

Activity 8 - Legacy System Case Study Discussion

Learning focus: real-world context

Purpose

Connect classroom techniques to industry practice.

Learning objectives targeted

- Describe legacy system challenges
- Explain why modernization is gradual

Team focus

Reflection, synthesis

Recommended group size

Pairs or whole class

Time estimate

15–25 minutes

Best for

Lecture-constrained time

Description

Students discuss claims from a legacy system case study and reflect on trade-offs.

Debrief prompts

- What surprised you?
- What would you prioritize in that situation?

Notes / Reflections

[illegible]

Activity 9 - Museum of Antiquated Code

Learning focus: perspective and curiosity

Purpose

Build appreciation for the diversity and longevity of software systems.

Learning objectives targeted

- Recognize that “legacy” spans many technologies

Team focus

Curiosity, humility

Recommended group size

Any

Time estimate

10–15 minutes

Best for

Lecture-constrained time

Description

Students examine short snippets of older languages or styles and infer behavior.

Debrief prompts

- How would you approach changing this?
- What skills transfer across eras?

Notes / Reflections

[illegible]

Week 10 - Measuring Team and Project Successes

Week overview

This week focuses on **how teams measure progress and adapt plans**, using evidence rather than intuition. Students practice interpreting simple project metrics, identifying what those signals imply about team behavior, and deciding what to change in response.

This week also provides structured time for **inspiration and synthesis**: showcasing prior years' projects and demo reels to help students calibrate scope, improve final presentation quality, and spark creativity for their own final submission.

Learning objectives

By the end of this week, students should be able to:

Project Execution

- Measure project completion progress
 - Predict and change existing project plans based on retrospective outcomes
-

Outside-of-class work

Readings

- The Path to QA Engineering: How I Got Here and Why I Stayed
- Charting Her Own Course: A Business Analyst's Story of Success
- My Developer Journey - Finding My First Job
- A Week in the Life of a Scrum Master
- First QA Engineer in a Startup
- How I Got My First Developer Job... 135 Applications Later!

Optional:

- A Conflict-Hating Scrum Master's Story
-

Pacing and sequencing notes (important)

This week is best run as a **synthesis week**:

- Some time is allocated to viewing past project examples and discussing what “success” looks like in deliverables and demos
- In-class activities should focus on reflection, metrics interpretation, and plan adjustment

Instructors should expect to run **2-3 activities total**, depending on how much showcase time is included.

Instructor notes

Students often:

- Treat metrics as performance evaluation rather than decision-support
- Confuse “busy” with “progress”
- Use retrospectives as venting rather than as planning tools

Keep the emphasis on: *What does the evidence suggest? What will we change next?*

Activity 1 - Project Showcase and Demo Reel Debrief

Learning focus: calibrating scope, recognizing quality, generating ideas

Purpose

Spark creativity and help teams develop a concrete sense of what strong final submissions look like.

Learning objectives targeted

- Measure progress toward a quality final submission
- Identify gaps between current work and desired outcome

Team focus

Reflection, aspiration, constructive comparison

Recommended group size

Whole class

Time estimate

20–40 minutes (depending on number of examples)

Best for

Lecture-constrained or activity-flexible time

Description

Show selected projects and demo reels from past years. After each, students note:

- One thing the project did well (clarity, polish, scope, demo narrative)
- One thing they would improve
- One idea they could borrow (without copying)

Optionally have teams discuss: “What does this suggest we should prioritize in our final iteration?”

Debrief prompts

- What separated the strongest demos from the weakest?
- What makes a project feel complete (even if small)?
- What should your team change this week after seeing these examples?

Notes / Reflections

Activity 2 - Burndown Chart Analysis Game

Learning focus: interpreting project signals, diagnosing process issues

Purpose

Teach students to interpret Agile metrics as evidence of team behavior and planning quality.

Learning objectives targeted

- Measure project completion progress
- Predict changes needed to project plans

Team focus

Sensemaking, diagnosis

Recommended group size

3–5 students

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams analyze example burndown charts (smooth burn, scope creep, end-loaded work, stalled progress). For each chart they infer:

- What likely happened
- What risk it suggests
- What the team should do next iteration

Teams share interpretations; instructor confirms or expands.

Debrief prompts

- What does this chart reveal about planning and execution?
- What is the smallest change that would improve the next iteration?

Notes / Reflections

Activity 3 - Team Retro - Start, Stop, Continue

Learning focus: actionable retrospectives

Purpose

Practice a structured retrospective that produces concrete action items.

Learning objectives targeted

- Predict and change project plans based on retrospective insights

Team focus

Honesty, accountability, improvement

Recommended group size

Permanent project teams

Time estimate

25–35 minutes

Best for

Activity-flexible time

Description

Teams identify:

- What to start doing next iteration
- What to stop doing because it harms progress
- What to continue doing because it helps

Teams must convert at least two items into explicit commitments (who will do what, by when).

Debrief prompts

- Which change would have the largest effect next week?
- What will you do differently starting immediately?

Notes / Reflections

Activity 4 - Agile Values Retrospective Sort

Learning focus: linking lived experience to Agile principles

Purpose

Help students evaluate how Agile their execution became, based on real experience.

Learning objectives targeted

- Predict changes to improve team execution and collaboration

Team focus

Reflection, constructive disagreement

Recommended group size

Permanent project teams

Time estimate

20–30 minutes

Best for

Lecture-constrained or activity-flexible time

Description

Teams sort Agile principles (or a simplified list of course-relevant values) along a spectrum from “we did this well” to “we struggled with this.” Discussion is the goal; teams should name one concrete example for two principles.

Debrief prompts

- Which principle was hardest to live out, and why?
- What process change would improve alignment next iteration?

Notes / Reflections

Activity 5 - Roles in Software Engineering Pictionary

Learning focus: professional roles, teamwork appreciation

Purpose

Lightly reinforce awareness of different software roles and how success is shared across them.

Learning objectives targeted

- Interpret project success as multi-dimensional (quality, collaboration, planning)

Team focus

Engagement, perspective

Recommended group size

Whole class or teams

Time estimate

15–20 minutes

Best for

Lecture-constrained time

Description

Students draw or act out roles (QA, Scrum Master, BA, DevOps, etc.) based on the week's readings. Class guesses and briefly discusses what "success" looks like in that role.

Debrief prompts

- Which role's success measures surprised you?
- What roles did your team implicitly rely on?

Notes / Reflections

[illegible]

Activity 6 - Kudos and Specific Appreciation Session

Learning focus: team health, recognition, closure

Purpose

Reinforce positive team practices and make contributions visible.

Learning objectives targeted

- Identify behaviors that contributed to team success

Team focus

Kindness, specificity, cohesion

Recommended group size

Teams or whole class

Time estimate

10–20 minutes

Best for

Lecture-constrained time

Description

Students write brief appreciations for teammates focused on specific actions (helping resolve conflicts, improving tests, clarifying requirements, consistent reviews). Notes may be shared privately within teams or optionally displayed.

Debrief prompts

- What behaviors most improved team functioning?
- What should future teams imitate?

Notes / Reflections
